

Real Time Digital Image Processing Using Point Operations in Multithreaded Systems

Samyan Q. W¹, Sahar W.², Talha W.³, Aslam M.⁴

Department of CS. & Engg., UET, Lahore, Pakistan

¹samyanwahla@gmail.com, ²saharwaqar@outlook.com,

³twahed@uet.edu.pk, ⁴maslam@uet.edu.pk,

Martinez-Enriquez A. M.

Department of Computer Science

CINVESTAV, D.F., Mexico

ammartin@cinvestav.mx

Abstract—This paper deals with real-time digital image processing using multi-threading for efficient and reliable real-time output. Two types of classes are dealt with: class A where processing is based on single pixel operations and class B where output is based on neighborhood pixel values, which is dedicated to convolution operations. Partitioning an image into sub-images to process it using multiple threads is not as simple as it looks. One issue is of information loss, which occurs during partition, especially at boundary pixels. Calculation of optimal size of sub-images is another issue. Whereas, writing code for gaining maximum performance via multi-threading is also cumbersome. In this paper, issue of how loss affects the speed of output with limited or desired number of resources is tackled. The aim is to reduce the information loss and improve performance. A generic multi-threaded API, to handle different image operations, is tested on images of different sizes and performance is compared with single-threaded approach. Experimental results demonstrate better execution time without data loss.

Keywords: Multi-threading, neighbourhood pixelpoint operations, image enhancement, convolution, real-time video and image processing.

I. INTRODUCTION

Image and video processing takes a lot of computational time and requires many system resources (like processor, memory, and storage) due to the computation of matrices of pixels [1]. The quest of getting real-time output from image processing functions is increasing as well. Multi-threading is one of the solution to improve performance of video and image processing. Multi-threading works on multiple requests in parallel, it is an effective way of performing any computation to get the output quickly and efficiently in offline videos and in real-time environment too.

Different types of image enhancements functions are common in image processing; they are used to improve visual appearance of image like: brightness, sharpness, noise removal, filters, and color adjustments for instance, in medical, aerial, and satellite images [3]. Even during output calculations, different filters need to be applied on images. These enhancements introduce delays in getting real-time output from the system [13]. They fall in following two classes:

In **class A**, point operations include image filters which are independent of pixels in the neighborhood. Enhancements in this class may include brightening the image, grey scaling, and adding contrast. In **class B**, neighborhood pixel operations are compromised with the set of filters in which the computation of value on a pixel involves some number of pixels in the

neighborhood [14]. Enhancement in this class includes filters like image smoothening, image sharpening, etc.

Processing a single frame, in addition to video, in multi-threaded manner is an issue that lies with processing a single image. Should the output of a pixel depend on itself or on the value of the function applied to the neighborhood pixels? The latter is more common than the former. Major filters and detection algorithms work on neighborhood pixels. While dealing with class B (neighborhood pixel) operations, information loss occurs on boundary pixels. When an image is multi-threaded with filters from class B, there are more issues with output combining and information loss, after splitting image into sub-images.

For increasing speed of the output, after applying image processing techniques, different researchers have focused on the multi-threading but, most of them have only focused on class A functions of images processing. This is pretty straight forward that if pixel value computation is independent of pixels in neighborhood then simply combine all the blocks after getting output from all the threads. Whereas, using digital image processing operations on class B, introduces the information loss due to boundary pixels.

In this paper, a solution is presented to reduce the information loss that occurs while partitioning the image into sub-images for sending them to different threads for fast processing. Proposed solution goes well with dividing images into sub-images or sub-blocks to send them in multiple threads and combining the results from these threads related to functions that either fall in class A or class B. Further, a generic application programming interface (API) based solution have been provided which is independent of image processing function implementation, i.e., multi-threading is abstract to API user.

The paper is organized as follows: Section 2 overviews the related work concerning multi-threading in image processing. Section 3 explains the class A point operations in image processing. The results of the multiple-thread based algorithms are discussed in Section 4. Conclusion and future work are given in Section 5.

II. RELATED WORK

Incorporating multi-threading in image processing has been a main focus since real-time video processing took hype. Various image processing algorithms were redesigned and implemented for using multi-threading on single and multi-core machines [5] and also on different operating system

platforms for comparison. However, the context always remained 'image'. For real-time video no useful work has been done so far. To get output from a video fast image has to be processed efficiently. This is the reason behind that image has been discussed more than video.

To deal with highly realistic graphical image extremely parallel, multi-threaded and multi-core architecture are introduced that perform real-time ray tracing. This architecture is called TraX [3], which provides read-only access to scene database for all threads. A simple architecture is developed at 500MHz, providing real-time ray tracing for scenes of complexity like video games, on a multi-core machine. TraX focuses at the architecture level which requires design changes at hardware level, but does not work on all platforms due to its rigidity towards multi-core machines [2].

Using four-thread model, a lock free multi-threading approach is developed to utilize multi-core processor efficiently [1]. In this approach, image data is partitioned into blocks, creating individual threads to handle the processing of each block independently. All outputs are combined into single module. Up to 71% computing performance is achieved from parallelization over sequential approach. In this work, only specific image algorithms are targeted.

Kamalakkannan gives image processing technique in multi-core CPU [4]. Cyganek added parallelism to the hybrid image processing library in multi-threading and multi-core systems [9]. Similarly, work also has been done related to multi-threading in image processing on low cost CPUs [10].

Mantiuk presented [6] highly dynamic range imaging pipeline using holistic approach whose accuracy depends on the hardware acceleration and causes computational overhead using multi-threading techniques.

Advanced Micro Devices (AMD) create a performance library which significantly optimize the techniques and reduced multi-threading burden. By using their library, they show that zooming of image is fastened using multi-threading techniques [7].

Grace [8] provides programmers with the appearance of deterministic and sequential execution while taking advantage of available processing cores to run code concurrently and efficiently. Grace, a software-only runtime system, eliminates concurrency errors for a class of multi-threaded programs: those based on fork-join parallelism. By turning threads into processes, leveraging virtual memory protection and imposing a sequential commit protocol [9].



Figure. 1 Image partitioning

Currently, multi-threading is being applied by researchers at all possible levels but none has come to a practical solution that could discuss effect of the algorithm in real-time applications. Whereas, for implementing any algorithms using multi-threading, user has to write threading logic in his code as

well, that is cumbersome. All these factors should be taken into account for future improvements in multi-threading.

In this paper, we tackle both class A and B type operations using multi-threading by extending libraries as generic such that user can employ our platform unknowingly algorithms in details.

III. GENERIC MULTI-THREADED IMAGE PROCESSING MACHINE

As discussed in Section I, image processing functions involve single point operations and convolution. To improve machine processing, different factors are taken into account:

- How many numbers of threads should be introduced to have fast results?
- How an image should be divided so that equal distribution of tasks may occur at all thread levels?
- How an image should be partitioned to lessen the information loss that occurs at boundary conditions?

One can apply multi-threading in image processing only by partitioning the image into sub-images and then sending each sub-image to an individual thread for processing. In this section, we first analyze how an image can be effectively partitioned limiting the number of threads to a proper N for fast performance. Then, we discuss multi-threading on each class and propose how user can be made independent of the logic in code he is implementing.

A. Image Partitioning with Equal Distribution

Partition of an image is in the form of vertical strips which behave as sub-images. Generic formula for breaking down image in N equal parts is given in Eq. 1.

$$\text{Size of strip} = S = \lceil W / N \rceil \quad (1)$$

Where W is the width of original image and N is number of strips in which original image is to be divided e.g. in Fig. 1, an image of resolution 1024x768 is broken down into 3 equal parts. Size of each strip is 342 pixels wide except the last strip which has size of 340 pixels.

Since, image sizes and resolutions are not consistent. For real-time video processing, it depends on the quality of camera and resolution of images it supports. In case of high definition cameras resolution increases, while for those used for surveillance compromise has to be made to reduce the cost, processing time of images and storage capacity. So, question is how to choose an optimal N ? As N is not in our control, because picture size and resolution depends on the user's choice, then it might have unequal distribution of images into sub-images. In our case, it deals with the number of threads that handle each sub-image; N is taken from the number of cores available in the machine.

If image can't be equally divided into sub-images of same width, we divide the width of image with N , round the result to the closest integer. To reduce the image loss, we track how much pixel information is being lost after calculating the size of strip. The delta (δ) is obtained by subtracting the size of strip from the actual width of the image as given in Eq. 2. Where, W is the width of the input image, N is the number of threads, and S is the width of the sub-image.

$$\delta = W - N*S \quad (2)$$

To deal with this non-uniform behavior of one strip, we split δ into one sub-unit each and add that one unit to all of δ times sub-images. As δ is always less than the number of sub-images, we can have a uniform behavior for all except for certain sub-images. But, now probability for non-uniform behavior is lesser than before. We can summarize δ in Eq. 3, where $\delta < N$.

$$\delta = W \text{ modulo } N \quad (3)$$

Splitting an image into N partitioning takes roughly constant time. As N is usually small and single digit as we partition image in place. This helps in saving memory for other tasks and efficiently reduces the load on the machine that is used if allocating other memory resources for handling single image only. So, size of strips in our example is approximately 342x768 pixels for a 1024x768 pixels image.

B. Multi-threaded Single Point Operations

Single-point processing is a simple method of image enhancement. This method of image enhancement determines the value of output pixel which is only dependent on the value of corresponding input pixel value. Examples of single point operations include thresh holding and negative of image [15]. Generic form of single point operations is represented by Eq. 4.

$$y = f(r) \quad (4)$$

Where, r is the single pixel input which contains the pixel value, f is the function which is applied on pixel r and y is the output in the form of single pixel.

Applying multi-threading on such type of operations is simple and does not involve many issues. The only issue that we have to deal with in this scenario is synchronization of output from different threads. Basic process for single point operations in multi-threading involves the layered process shown in Fig. 2.

The proposed approach works as follow; first of all image is acquired through image acquisition methods whether it is captured from a video stream directly connected to camera as in the real time environment or from a recorded video. After image is acquired in the first layer then image partitioning is performed on the input image using the algorithm suggested in sub-section A which divides the images into the vertical strips of the smaller images.

Each vertical strip is assigned to a thread as a task from thread pool. Performance and efficiency is the first preference in the suggested approach, for which instead of thread creation we use thread pool which contains the idle threads available for processing. Each thread evaluates the point processing operation on its assigned strip and generated a processed vertical strip.

After completion of tasks from each thread, output from each thread is synced and final processed image is available as output.

The uniqueness in our designed approach to handle single point operations is that we provide the flexibility to user to write their own single point operations without sticking in the multi-threaded logic. Our uniform API (discussed in sub-section D) automatically injects this logic in the user functions. After all threads finish their tasks, their output is automatically synchronized in the output matrix. The proposed architecture is

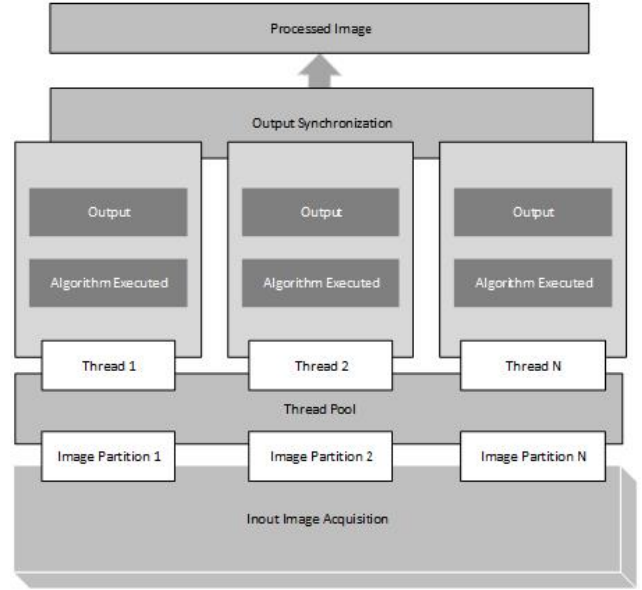


Figure. 2 Layered process for single point image processing

efficient in code writing, execution, and output synchronization.

C. Multi-threaded Convolution

Convolution is a typical image processing function and central to modern image processing that progression the intensity of a pixel to mirror the intensities of the neighborhood pixels. There are multiple image enhancement techniques that uses convolution including image sharpening and image blurring [11]. The basic idea is that a finite size and shape window (known as kernel) is scanned across the image. Utilizing kernels to perform convolutions is known as kernel convolution. Equation 5 gives convolution calculations.

$$c[m,n] = a[m,n] * h[m,n] \\ = \sum_{j=0}^{J-1} \sum_{k=0}^{K-1} h[j,k] * a[m-j,n-k] \quad (5)$$

Large images accordingly require more convolution than the same operation on a small image. Convolution usually take more time than other image processing operations, as it involves a lot of mathematical operations, and therefore our presented work is more focused on convolution operations.

While dividing image into sub-images we can't calculate convolution on boundary pixels. To handle this problem, there is some extra processing required to handle missing pixels when applying the kernel. There are multiple methods to handle boundaries in an image including extending, wrapping, or cropping. All of these methods make the calculated values less confident as boundary pixels are assumed.

When, we handle edges and corner pixels in the input image there emerge some non-reliable information on the edges as we are extending the image logically and conceptually and adding some information from ourselves. Fig. 3(a) shows the case of 3x3 kernel for an input image in which edge information is not reliable. Unreliable pixels are shown in grey color. Number of affected pixels in any input image of size $m \times n$ and kernel size of $j \times k$ can be calculated using Eq. 6. Where,

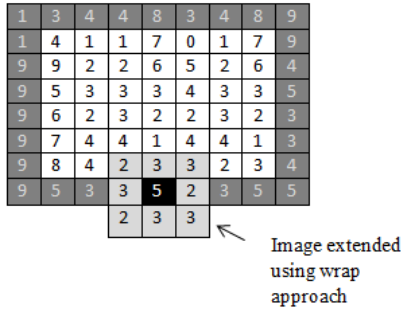
m and n are height and width of input image and w is the number of affected pixels. And j and k are the height and width for kernel grid. These problems make multi-threading for convolution operations almost useless and also cause information loss for given images. In our work, it is handled by proposing to share data of adjacent vertical strips. Any thread x may have read access of image matrices of adjacent threads $x-l$ and $x+l$ as shown in Fig. 4.

$$w = \left[2m \times \frac{k-1}{2} + 2n \times \frac{j-1}{2} \right] - \left[\frac{k-1}{2} \times \frac{j-1}{2} \right] \times 4 \quad (6)$$

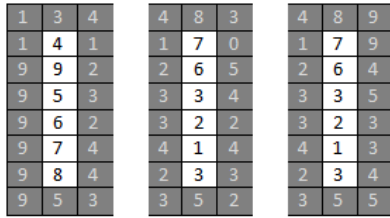
Number of affected pixels in an input image increases with the increase of kernel size. When the image is partitioned for multi-threading using approach discussed in sub-section A, number of affected pixels also increases as shown in the Fig.3 (b). Original image matrix is divided into 3 vertical strips. It is obvious from the Fig. 3(b) that number of affected pixels is directly proportional to number of strips which is represented with N in Eq. 7.

$$W \propto N \quad (7)$$

We propose the architecture for multithreading in convolution similar to the approach for single point operations with some additions to our layered approach which is shown in Fig 4. Convolution operations are complex and they need information sharing among threads. Without of information sharing among threads, problems of information reliability and loss of details are faced.



(a) Before Partitioning



(b) after partitioning

Figure. 3 Border pixels in convolution

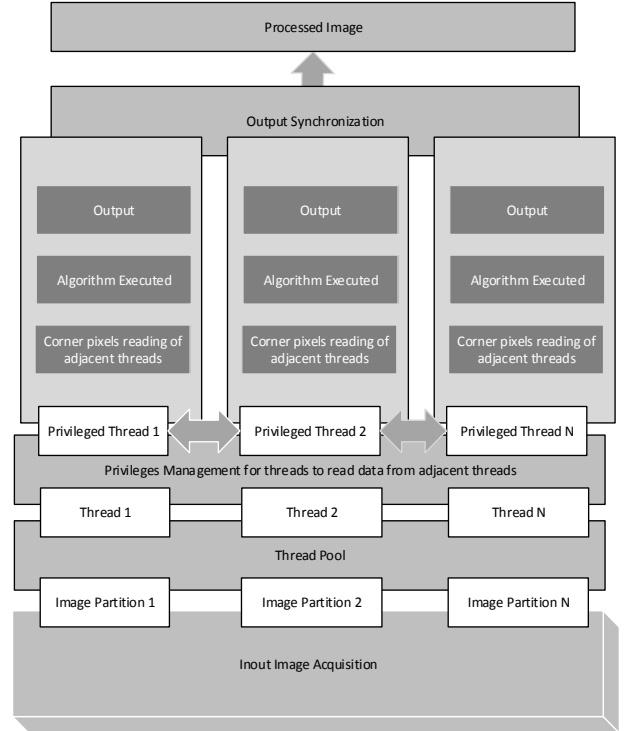


Figure. 4 Layered process for convolution multithreading

After acquiring the image from some image acquisition source, it is passed to image partitioning algorithm to divide it into vertical strips, each vertical strips is then assigned a thread from the thread pool. After thread assignment to sub images we add a layer of privileges management for threads. Threads can access their own data only but in convolution operations we need to read data from adjacent thread so that we minimize the data loss. Privileges of data reading from adjacent threads are assigned which allows threads to read corner pixels of adjacent images. Image processing operation is executed in each thread which the input of corner pixels from adjacent strips. After completion of task by each thread, output is synchronized and processed image is available.

D. Generic API to Handle Image Operations

Presented work is focused to handle all image processing functions transparent to API users and developers of image processing operations. Fig. 5 shows the architecture developed for generic multi-threaded API. The API deals with all those algorithms which involve convolution and single point operations. The API automatically handles the issues of information loss and synchronization which are faced when convolution algorithms are divided into different threads.

Other APIs which are focused on point operations of image processing provides some commonly used algorithms in which they have implemented multi-threading and then have exposed these algorithms through an interface. Our generic multi-threaded API accepts any customized algorithm; user just has to write an algorithm without messing up with multi-threading logic. Our system automatically inject multi-threading in provided algorithm.

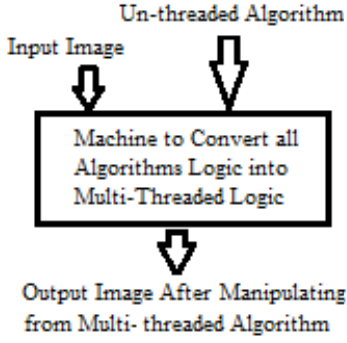


Figure. 5 Generic API architecture

IV. EXPERIMENTAL RESULTS

Three algorithms brightness, sharpness and median filter have been implemented by MATLAB image processing library [12] which is used on Windows operating system after injecting multithreaded logic and each one executed 15 times. Brightness is single point operation while sharpen and median filters are convolution operations. The results of average execution time without multi-threading on all four image sizes are shown in Table I. Then, all these algorithms were executed with multi-threading logic by using three and ten threads. Average results of these experiments are shown in Table II.

The execution time of algorithm is dependent on size of input image as well. Experiments are performed using four different images of size; 200x200, 400x400, 600x600 and 1024x768. In our results these dimensions are referred as Category 1, Category 2, Category 3, and Category 4. The size of kernel in convolution is also an important consideration; in these experiments, kernel size is 3x3.

TABLE I. PERFORMANCE OF IMAGE PROCESSING ALGORITHMS WITHOUT MULTI-THREADING

Image Size	Execution time (ms)		
	Sharpness	Brightness	Median Filter
Category 1	97	20	98
Category 2	225	37	289
Category 3	297	62	345
Category 4	482	85	589

TABLE II. PERFORMANCE OF IMAGE PROCESSING ALGORITHMS

Image Size	Execution time (ms)					
	Sharpness		Brightness		Median Filter	
	3 threads	10 threads	3 threads	10 threads	3 threads	10 threads
Category 1	45	92	8	18	56	96
Category 2	108	215	19	32	126	259
Category 3	158	298	29	61	198	309
Category 4	204	485	36	82	201	550

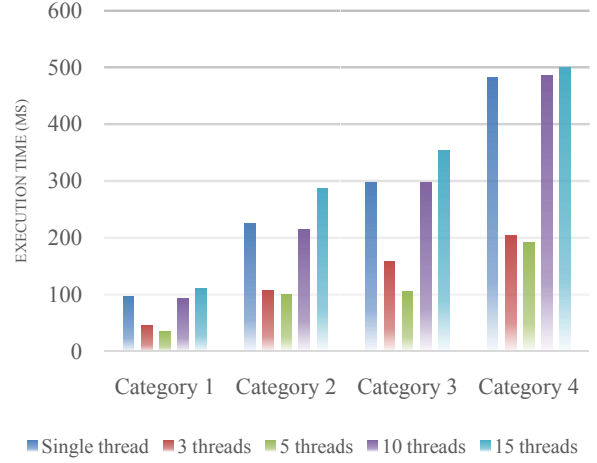


Figure. 6 Results of changing number of threads for sharpening filter

Result in Table II shows that increasing the number of threads enhances the performance up to certain level, after that further increasing the number of threads reduces the performance and increase the execution time. Greater number of threads is better for the machines having greater number of CPU cores. As Fig.6 shows that for a particular category in case of sharpening filter, there is a performance peak point which in our case is for five threads but increasing the number of threads from 3 to 15 threads increases the execution time and performance decreases to almost the approach without multi-threading. The performance peak point is dependent of CPU processing power and is different in case of different number of cores in the CPU.

Multiple approaches are there which give us performance and efficiency but the uniqueness of our results is that these results are obtained from generic multi-threaded API which gives user the independence of implementation of multi-threaded logic which is not provided by any existing implementation. The results of these filters are exactly the same as those without multi-threaded logic which infer no data loss in the case of convolution.

Generic multithreading API helps to inject multithreading logic in any algorithms without the workload of rewriting the existing algorithms and worrying about to make algorithm stable in case of complex data sharing among and data synchronization, designing of complex algorithms is made simpler and faster.

V. CONCLUSION AND FUTURE WORK

Multi-threaded machine is tested and verified on Windows operating system for the single point operations and convolution operators with four image sizes. Performance can be improved further to test multi-threaded API on different operating system platforms like Linux, Solaris and, MAC OS and on different hardware platforms like Intel, AMD, NVidia. A lot of work has been done in the image processing domain but there are the scenarios in which real time processing is needed which includes working cameras at some public place like hotels, airports, hospitals and surveillance cameras. In such type of scenarios, a single frame is bottleneck for researchers to

work in real time environment which takes a lot of seconds to process an image and generates an output, this work opens the research opportunities for multi-threading in video processing combined with single frame image processing which will help image processing field to gap the performance issues which will help researchers to design their algorithms in the real time environment.

VI. REFERENCES

- [1] Carlston, P., "Optimizing digital signal and image processing on Intel architecture processors", Intel Corporation, White Paper, Jan 2009.
- [2] Patil, T., "Evaluation of multi-core architectures for image processing algorithms", graduate thesis, Clemson University, Aug 2009.
- [3] Spjut, J., Boulos, S., Kopta, D., Brunvand, E., and Kellis, S., "TRaX: A multi-threaded architecture for real-time ray tracing", SASP'08, Anaheim CA, Jun 2008. pp.108-114.
- [4] Kamalakannan, and A., Rajamanickam, G., "High performance color image processing in multicore CPU using MFC multithreading", Int. J. of Adv. Computer Science and Applications (IJACSA), Vol. 4, No.12, Dec 2013. pp. 42-47.
- [5] Kika, A., and Greca, S., "Multithreading image processing in single-core and multi-core CPU using Java," Int. J. of Adv. Computer Science and Applications (IJACSA), Vol.4, No.9, 2013. pp. 165-169.
- [6] Mantiuk, R., and Pajak, D., "Acceleration of high dynamic range imaging pipeline based on multi-threading and SIMD technologies", Computational Science – ICCS 2008, 8th Int. Conf., Karkow, Poland, Jun 23-25, 2008. Chapter in Proc, (Ed.) M. Bubak, G. D. Albada, J. Dongarra, P. M. A. Sloot, Part I, LNCS 5101, Springer 2008, pp. 780-789.
- [7] Hollingsworth B., "Image processing the easy way: image zoom with and without AMD performance library", Advanced Micro Devices Inc., 2006.
- [8] Berger E. D., Yang, T., Liu, T., and Novark. G., "Grace: safe multithreaded programming for C/C++", Newsletter ACM SIGPLAN Notices - OOPSLA 2009, Vol. 44(10), Oct 2009, ACM, NY, USA. pp.81-96.
- [9] Cyganek, B., "Adding parallelism to the hybrid image processing library in multi-threading and multi-core systems", IEEE 2nd Int. Conf. on Networked Embedded Systems for Enterprise Applications (NESEA), Dec 8-9, 2011, WA, USA.
- [10] Shah, S., "Real-time image processing on low cost embedded computers", Technical Report UCB/EECS-2014-117, UC Berkeley, May 20, 2014.
- [11] Bedi, S. S., and Khandelwal, R., "Various image enhancement techniques- a critical review," Int. J. of Adv. Research in Computer and Communication Engineering, Vol. 2, No. 3, Mar 2013. pp.1605-1609.
- [12] MathWorks, "Enhancement methods in image processing", Page from "Matlab Tutorial – Discover how to solve your computational problem", [Online] <http://www.mathworks.com/discovery/image-enhancement.html> [Accessed Jul 14, 2015].
- [13] Vermon, D., "Fundamentals of digital image processing", chapter in "Machine vision - automated visual inspection and robot vision", Prentice Hall, 1991, ISBN 0-13-543398-3.
- [14] Sahoo, P.K., Soltani S., Wong, A.K.C., and Chen Y.C., "A survey of thresholding techniques", J. of Computer Vision Graphics and Image Processing, Vol.41, No. 2, Academic Press, CA, USA, 1988, pp. 233-260.
- [15] Fisher, R., Perkins, S., Walker, A., and Wolfart, E., "Point Operations", Page from Hypermedia Image Processing Reference (HIPR2), [Online] <http://homepages.inf.ed.ac.uk/rbf/HIPR2/pntops.htm> [Accessed May 2, 2015].